1 Introduction to Data Structures & Algorithms (DS&A)

Data Structures (DS) are ways to organize and store data efficiently for easy access and modification. Algorithms (Algo) are step-by-step procedures or formulas for solving problems, often using these data structures. Together, DS&A form the foundation of computer science and software engineering, enabling efficient problem-solving in areas like software development, machine learning, databases, and more.

1.1 Why Learn DS&A?

- **Efficiency**: Optimize time and space usage (e.g., searching a list in $O(1)$ vs. $O(n)$ time).
- **Problem-Solving**: Essential for coding interviews (e.g., at FAANG companies), competitive programming (e.g., LeetCode, Codeforces), and real-world applications.
- **Scalability**: Handle large datasets (big data) without performance degradation.
- **Core Concepts**: Understanding trade-offs like time vs. space complexity.

DS&A are language-agnostic but implemented in languages like Java, Python, or C++. I'll use Java examples here for consistency with prior discussions, but concepts apply universally.

2 Complexity Analysis

Before diving in, understand how to measure efficiency using **Big O Notation** (asymptotic analysis), which describes worst-case performance as input size (n) grows.

2.1 Key Notations

- **Time Complexity**: Operations as a function of n.
- **Space Complexity**: Memory usage.

Notation	Description	Example Use Case
$O(1)$	Constant time	Array access by index
$O(\log n)$	Logarithmic	Binary search
$O(n)$	Linear	Traversing a list
$O(n \log n)$	Linearithmic	Merge sort
$O(n^2)$	Quadratic	Nested loops (bubble sort)
$O(2^n)$	Exponential	Recursive Fibonacci
$O(n!)$	Factorial	Permutations

Table 1: Common Big O Notations

- **Best/Average/Worst Case**: Always focus on worst-case for reliability.
- **Amortized Analysis**: Average cost over operations (e.g., ArrayList resizing).

Tip: Visualize with graphs—$O(1)$ is flat, $O(n)$ is linear, $O(n^2)$ grows steeply.

3 Fundamental Data Structures

These are building blocks for more complex ones.

3.1 Arrays

- **Description**: Fixed-size, contiguous memory blocks storing elements of the same type. Indexed access (0-based).
- **Pros**: $O(1)$ access, cache-friendly.
- **Cons**: Fixed size (in some languages), $O(n)$ insertion/deletion (shifting elements).
- **Java Example**:

```java
int[] arr = new int[5]; // Fixed size 5
arr[0] = 10; // Set value
System.out.println(arr[0]); // Access: 10
```

- **Variants**: Dynamic arrays (e.g., Java's ArrayList) resize automatically.

3.2 Linked Lists

- **Description**: Nodes with data and a pointer to the next node. No fixed size.
- **Types**:
  - Singly Linked: One-way traversal.
  - Doubly Linked: Two-way (prev/next pointers).
  - Circular: Last node points to first.
- **Pros**: $O(1)$ insertion/deletion (if pointer known), dynamic size.
- **Cons**: $O(n)$ access (no random access), extra space for pointers.
- **Java Example** (Singly Linked List Node):

```java
class Node {
    int data;
    Node next;
    Node(int data) { this.data = data; }
}

Node head = new Node(1);
head.next = new Node(2); // Link nodes
```

3.3 Stacks

- **Description**: LIFO (Last In, First Out) structure. Operations: push (add), pop (remove), peek (top element).
- **Use Cases**: Undo/redo, recursion (call stack), parentheses matching.
- **Implementation**: Array or Linked List.
- **Pros/Cons**: $O(1)$ operations; limited access.
- **Java Example** (Using Stack class):

```java
import java.util.Stack;
Stack<Integer> stack = new Stack<>();
stack.push(1); // Add
int top = stack.pop(); // Remove and get: 1
```

3.4 Queues

- **Description**: FIFO (First In, First Out) structure. Operations: enqueue (add to rear), dequeue (remove from front), peek.
- **Use Cases**: Task scheduling, BFS in graphs, buffers.
- **Variants**: Priority Queue (elements ordered by priority), Deque (double-ended queue).
- **Implementation**: Array (circular for efficiency) or Linked List.
- **Pros/Cons**: $O(1)$ operations; array-based may waste space.
- **Java Example** (Using LinkedList as Queue):

```
import java.util.LinkedList;
import java.util.Queue;
Queue<Integer> queue = new LinkedList<>();
queue.add(1); // Enqueue
int front = queue.poll(); // Dequeue: 1
```

3.5 Hash Tables (Maps)

- **Description**: Key-value pairs using hashing for $O(1)$ average-case access. Handles collisions via chaining or open addressing.
- **Pros**: Fast lookups, insertions, deletions.
- **Cons**: Worst-case $O(n)$ if many collisions; space overhead.
- **Java Example**:

```
import java.util.HashMap;
HashMap<String, Integer> map = new HashMap<>();
map.put("apple", 5);
int value = map.get("apple"); // 5
```

4 Advanced Data Structures

4.1 Trees

- **Description**: Hierarchical structure with nodes (root at top), edges, no cycles.
- **Types**:
  - **Binary Tree**: Each node ≤ 2 children.
  - **Binary Search Tree (BST)**: Left child $<$ parent $<$ right child; $O(\log n)$ search/insert/delete.
  - **Balanced Trees**: AVL, Red-Black (self-balancing for $O(\log n)$ guarantee).
  - **Heaps**: Complete binary tree; Min-Heap (smallest at root) or Max-Heap. Used for priority queues.
- **Traversals**: In-order (left-root-right), Pre-order (root-left-right), Post-order (left-right-root), Level-order (BFS).

- **Use Cases**: File systems, databases (indexes), Huffman coding.
- **Java Example** (BST Node):

```java
class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) { this.val = val; }
}
```

4.2 Graphs

- **Description**: Nodes (vertices) connected by edges. Can be directed/undirected, weighted/unweighted.
- **Representations**: Adjacency List (efficient), Adjacency Matrix (space-heavy).
- **Algorithms**: DFS (Depth-First Search), BFS (Breadth-First Search), Dijkstra (shortest path), Bellman-Ford, Floyd-Warshall, Topological Sort (for DAGs), MST (Minimum Spanning Tree: Kruskal, Prim).
- **Use Cases**: Social networks, maps (routing), recommendation systems.
- **Pros/Cons**: Flexible; can be complex (cycles, disconnected components).
- **Java Example** (Adjacency List):

```java
import java.util.ArrayList;
ArrayList<ArrayList<Integer>> graph = new ArrayList<>();
for (int i = 0; i < 5; i++) graph.add(new ArrayList<>()); // 5 nodes
graph.get(0).add(1); // Edge 0 -> 1
```

4.3 Tries (Prefix Trees)

- **Description**: Tree for storing strings, each node a character. Efficient for prefix-based searches.
- **Use Cases**: Autocomplete, spell checkers.
- **Pros**: $O(m)$ time where m is key length.

4.4 Disjoint Set Union (DSU/Union-Find)

- **Description**: Tracks partitions of sets; operations: find (parent), union (merge).
- **Use Cases**: Kruskal's MST, cycle detection.

5 Algorithms Categories and Examples

5.1 Sorting Algorithms

Java Quick Sort Example (Pseudocode-like):

Algorithm	Time Complexity (Worst)	Space	Stable?	Description
Bubble Sort	$O(n^2)$	$O(1)$	Yes	Repeated swaps of adjacent elements.
Insertion Sort	$O(n^2)$	$O(1)$	Yes	Builds sorted array one item at a time.
Selection Sort	$O(n^2)$	$O(1)$	No	Finds min and swaps to front.
Merge Sort	$O(n \log n)$	$O(n)$	Yes	Divide-conquer-merge.
Quick Sort	$O(n^2)$ (avg $O(n \log n)$)	$O(\log n)$	No	Pivot-based partitioning.
Heap Sort	$O(n \log n)$	$O(1)$	No	Builds heap then extracts max/min.

Table 2: Sorting Algorithms Comparison

```
void quickSort(int[] arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

5.2 Searching Algorithms

- **Linear Search**: $O(n)$, scan sequentially.
- **Binary Search**: $O(\log n)$, on sorted arrays; divide and conquer.

```
int binarySearch(int[] arr, int target) {
    int low = 0, high = arr.length - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
```

5.3 Recursion

- **Description**: Function calls itself. Base case prevents infinite loop.
- **Use Cases**: Factorial, Fibonacci, tree traversals.
- **Pros/Cons**: Elegant; stack overflow risk for deep recursion.
- **Memoization**: Cache results to optimize (e.g., Fibonacci).

5.4 Dynamic Programming (DP)

- **Description**: Break problems into subproblems, store results (bottom-up or top-down with memoization).
- **Use Cases**: Fibonacci ($O(n)$ vs. exponential), Knapsack, Longest Common Subsequence.
- **Example**: 0/1 Knapsack for max value with weight limit.

5.5 Greedy Algorithms

- **Description**: Make locally optimal choices for global optimum.
- **Use Cases**: Coin change, Fractional Knapsack, Huffman coding.
- **Pros/Cons**: Simple, fast; not always optimal (e.g., fails for some TSP variants).

5.6 Backtracking

- **Description**: Try all possibilities, backtrack on failure.
- **Use Cases**: N-Queens, Sudoku, permutations.

5.7 Divide and Conquer

- **Description**: Divide problem, solve subproblems, combine.
- **Use Cases**: Merge Sort, Binary Search.

6 Common Problem Patterns

- **Two Pointers**: For sorted arrays (e.g., find pair summing to target).
- **Sliding Window**: For substrings/subarrays (e.g., max sum of k elements).
- **Fast & Slow Pointers**: Detect cycles in linked lists.
- **BFS/DFS**: Graph/tree traversal.
- **Topological Sort**: Dependency resolution.
- **Union-Find**: Connected components.
- **Bit Manipulation**: Efficient operations (e.g., XOR for single number in duplicates).

Practice: Solve problems categorized by patterns on LeetCode.

7 Implementation Tips and Best Practices

- **Choose Right DS**: Arrays for random access, Linked Lists for frequent inserts, HashMaps for lookups.
- **Handle Edge Cases**: Empty inputs, single elements, maximum sizes.
- **Optimize**: Reduce space (in-place algorithms), use appropriate complexity.
- **Testing**: Write unit tests for various inputs.
- **Languages**: Java for OOP implementations; Python for simplicity.
- **Common Pitfalls**: Off-by-one errors, null pointers, overflow (use long for large numbers).

8 Applications in Real World

- **Databases**: B-Trees for indexing.
- **Web**: Graphs for page ranking (PageRank algorithm).
- **AI/ML**: Heaps for k-nearest neighbors.
- **Games**: A* search for pathfinding.
- **Systems**: Queues in OS scheduling.